CONTROL EDGE | EPISODE 2

Inside the Industrial Controller

PLC, PAC and Industrial PC architecture, timing, memory and I/O

Audience	Mechanical, process, manufacturing and automation engineers
Target duration	40-44 minutes
Format	Dialogue between Yael, a young mechanical engineer, and Amir, a senior process and controls engineer
Core example	A servo-indexed filling and circulation machine that combines motion, discrete sequencing and process control
Episode position	Episode 2 of the Control Edge series

Educational material. It does not replace engineering design, risk assessment, manufacturer instructions, or the contractually applicable editions of relevant standards.

Master-plan check
Episode 1 promised to open the control cabinet. This episode therefore stays below HMI and SCADA and focuses on the controller itself: hardware, execution, memory, I/O and availability. It introduces IEC 61131-3 only as the programming framework; detailed language design, coding conventions and testing remain for later episodes.

1. Pre-production alignment check

- The audience remains mechanical and process engineers, not only automation programmers.
- The episode deepens the controller layer without jumping prematurely to HMI, SCADA or cloud systems.
- The worked example combines motion, discrete logic and process control to connect the audience disciplines.
- Standards are presented as professional frameworks without invented clauses or claims of contractual applicability.
- The closing prepares Episode 3 on sensors and signal integrity.

2. Episode objectives

- Explain what makes an industrial controller different from an ordinary computer.

- Distinguish PLC, PAC, industrial PC, soft PLC, DCS controller and safety PLC without relying on marketing labels.
- Describe controller hardware: power supply, CPU, memory, backplane, communications and local or remote I/O.
- Explain scan cycles, cyclic and event tasks, priorities, interrupts, watchdogs, jitter and end-to-end response time.
- Describe volatile, non-volatile and retentive data and the engineering consequences of restart behaviour.
- Introduce the 2025 fourth edition of IEC 61131-3 and the roles of ST, LD, FBD and SFC.
- Build a practical controller-selection checklist that includes performance, environment, lifecycle, cybersecurity and maintainability.

3. Timing and segment plan

Time	Segment	Purpose
00:00-02:30	Cold open: the reject gate that was late	Show why average CPU speed is not the same as deterministic control.
02:30-05:30	What counts as a controller?	Define PLC, PAC, IPC, soft PLC, DCS and safety PLC boundaries.
05:30-10:00	Inside the hardware	Map CPU, memory, backplane, power, communications and I/O.
10:00-15:30	The execution model	Explain scan cycles, tasks, priorities and interrupts.
15:30-19:30	Determinism and watchdogs	Connect cycle time, jitter and latency to physical response.
19:30-24:00	Local and remote I/O	Explain update paths, diagnostics and failure behaviour.
24:00-28:00	Memory and restart states	Separate volatile, non-volatile and retentive data.
28:00-32:00	IEC 61131-3 and engineering tools	Introduce languages, POUs, online change and portability limits.
32:00-36:00	PLC versus PAC versus IPC	Select architecture by requirements rather than label.
36:00-39:30	Redundancy and high availability	Explain what is and is not duplicated.
39:30-42:00	Safety and cybersecurity	Separate basic control, FS-PLC

Time	Segment	Purpose
	boundaries	and secure operation.
42:00-44:00	Worked task map and closing checklist	Turn the concepts into a reusable engineering method.

4. Controller architecture decision map

There is no single strict standards boundary between PLC and PAC. Use this table to translate family names into requirements that can be measured, tested and maintained.

Architecture	Typical strengths	Questions before selection
Compact or modular PLC	Rugged deterministic logic, direct I/O, long industrial lifecycle, familiar maintenance workflow	Required I/O count, task periods, environmental class, communications, expansion and service support?
PAC	Vendor term commonly used for larger, data-rich, multi-domain controller platforms	What measurable capability is meant: memory, multitasking, redundancy, motion, process libraries or networking?
Industrial PC with real-time runtime	High compute density, open software integration, advanced vision, analytics and multi-core scaling	How is real-time isolated from the host OS? What is the recovery, patching and storage strategy?
DCS controller	Integrated process libraries, engineering database, alarms, historian and plant-wide lifecycle	Does the project need unit-wide process orchestration and high availability more than machine-cycle specialization?
Safety PLC / FS-PLC	Certified safety-related logic subsystem with controlled tools and diagnostics	What safety functions, integrity targets, independence, proof testing and lifecycle evidence are required?

Engineering caution
A controller label never proves real-time performance, safety integrity, availability or cybersecurity. Verify the complete platform, configuration, firmware, I/O path and application under worst-case conditions.

5. NotebookLM production prompt

Paste-ready instruction

Create an English-only audio episode of 40-44 minutes using only the uploaded source pack and this document. Use two hosts: Yael, a young mechanical engineer who asks practical design and maintenance questions; and Amir, a senior process and controls engineer who explains architecture, timing and lifecycle trade-offs. Make the exchange feel like two engineers reviewing a real machine, not a lecture read by alternating voices.

Use the servo-indexed filling and circulation machine as the continuous example. Give Yael roughly 45% of the speaking time and Amir 55%. Yael should challenge vague terms such as “fast PLC,” “PAC,” “real time,” “redundant” and “fail safe.” Amir should answer with measurable engineering quantities, failure modes and verification methods.

Expand every acronym the first time it appears. Distinguish international standards from vendor-specific implementation. State explicitly that example task periods, thresholds and architectures are illustrative, not design values. Do not invent standard clause numbers, certifications, product performance or vendor capabilities that are not present in the source pack. Do not claim that IEC 61131-3 guarantees source-code portability between vendors.

The episode must cover controller hardware, I/O paths, cyclic and event tasks, task priority, watchdogs, jitter, end-to-end response time, memory retention, restart behaviour, IEC 61131-3 languages, PLC/PAC/IPC selection, redundancy, safety-controller boundaries and controller cybersecurity. End with a seven-question selection checklist and a clear transition to Episode 3 on sensors and signal integrity.

6. Full episode script

00:00-02:30 | Cold open - the reject gate that was late

Amir: Picture a high-speed filling machine. A vision sensor detects a container with the wrong cap. The controller has sixty milliseconds to move a pneumatic reject gate before that container passes the diverter. During commissioning it works. Two months later, after a historian connector and a new diagnostic routine are added, one container in a few thousand escapes the reject station.

Yael: The immediate accusation will be that the PLC became too slow.

Amir: Yes, but “too slow” is not yet an engineering diagnosis. Was the input sampled late? Did a low-priority task delay the decision? Was the output image updated only at the end of another cycle? Did the network add jitter? Did the valve take longer because air pressure fell? Or did the cylinder mechanics simply have less margin than the software team assumed?

Yael: So processor speed alone does not answer whether the machine responds on time.

Amir: Exactly. Control timing is an end-to-end chain: event, sensor, input module, communication, task scheduling, logic, output update, actuator and mechanics. Today we open the controller and learn how that chain is organized.

Yael: And by “open” we mean conceptually. No energized cabinet work without the correct procedure.

Amir: A useful first interlock for the podcast.

02:30-05:30 | What counts as an industrial controller?

Yael: In drawings I see PLC, PAC, IPC, DCS controller, motion controller and safety PLC. Are these fundamentally different computers?

Amir: Sometimes the hardware overlaps. The more useful question is what execution guarantees, interfaces, environment and lifecycle the product is designed to provide. A programmable logic controller, or PLC, is a digitally operating industrial controller intended to execute user logic and control machines or processes through inputs and outputs. IEC 61131-1 provides the general framework, and IEC 61131-2 covers functional and electromagnetic-compatibility requirements and tests for industrial control equipment, explicitly including PLC and PAC products.

Yael: That already tells us something important: industrial controller is not just a small desktop computer in a metal box.

Amir: Right. It is designed around industrial power, temperature, vibration, electromagnetic disturbance, module diagnostics, controlled restart and long support cycles. A Programmable Automation Controller, or PAC, is a widely used industry label for platforms with larger memory, richer networking, multi-tasking, motion, process libraries or redundancy. But there is no single border where a PLC becomes a PAC. Treat the label as a prompt to inspect measurable capabilities.

Yael: And an Industrial PC?

Amir: An IPC is a rugged computer platform. It may run Windows, Linux, a real-time operating system or a vendor runtime. With a soft PLC, the IEC control runtime is software executing on that platform. PC-based control can combine PLC logic, motion, vision, analytics and HMI on one machine. The engineering question is how the real-time workload is isolated from non-real-time software, how storage and updates are managed, and what happens when the host operating system misbehaves.

Yael: A DCS controller is then selected less as a standalone box and more as part of a Distributed Control System.

Amir: Exactly. A DCS often brings an integrated engineering database, process-control libraries, alarm management, historian links and redundancy patterns across a plant. A safety PLC is different again: it is intended as the logic subsystem of a safety-related system and is governed by a safety lifecycle, certified toolchain and defined diagnostics. Similar enclosure, very different claim.

Yael: So our first rule is: ignore the badge for a moment and write the requirements.

Amir: Response time, I/O, environment, availability, communications, safety role, cybersecurity, maintainability and expected life. Then compare architectures.

05:30-10:00 | Inside the controller hardware

Yael: Let us take the cabinet door off in our mental model. What is inside a modular PLC rack?

Amir: Usually a power supply, a CPU or controller module, local input and output modules, and communication or specialty modules. A backplane or base connects them electrically and carries data. Some compact controllers put most of this into one housing. Larger platforms distribute it across racks and remote I/O stations.

Yael: The power supply sounds ordinary, but it can define system behaviour.

Amir: Absolutely. Ask whether the controller has hold-up time through a short voltage dip, how brownout is detected, what diagnostic event is generated, and whether separate power domains exist for logic, field inputs and outputs. A CPU may continue executing while an output supply has failed. If the program only looks at its own command bit, the HMI can say “valve open” while the solenoid has no power.

Yael: Then the CPU.

Amir: The CPU executes the runtime, schedules tasks, manages memory, diagnostics and communication. Its specification may include instruction execution benchmarks, application memory, communication capacity, number of axes, task limits and security functions. Do not compare controllers using one benchmark number. Boolean logic, floating-point control, communication load, motion interpolation and data handling stress different resources.

Yael: Memory is not one bucket either.

Amir: Correct. There may be load memory that stores the project, work memory used during execution, non-volatile storage, retentive areas, buffers and removable media. The exact names are vendor-specific, but the engineering questions are universal: what survives loss of power, what survives a warm restart, what is reinitialized, and what must be restored from a recipe or higher-level system?

Yael: Communication modules can also contain processors.

Amir: Yes. A controller may have integrated Ethernet and fieldbus interfaces plus separate modules for serial, motion, safety or remote I/O. Some communication is cyclic and tightly scheduled. Some is explicit or acyclic. Some happens in the CPU, some in dedicated hardware. That matters because heavy HMI polling or file transfer can compete with other work if the architecture is not designed to isolate it.

Yael: And specialty I/O?

Amir: High-speed counters, encoder interfaces, time-stamped event inputs, thermocouple modules, strain-gauge modules and motion interfaces. They often perform conversion or capture in hardware because the main task cannot reliably observe a microsecond event by polling every ten milliseconds.

Yael: So the rack is not a passive terminal strip. It is a distributed computing system in miniature.

Amir: That is a very good way to see it.

10:00-15:30 | Scan cycle, tasks and priorities

Yael: Every introductory PLC course shows the scan: read inputs, execute program, update outputs, repeat. Is that still true?

Amir: It is a useful first model, not a complete description. Many controllers maintain an input image, execute logic and update an output image. But modern platforms can run multiple tasks: continuous, periodic, event-triggered or synchronized to an I/O bus or motion cycle. Rockwell, for example, organizes execution through tasks, programs and routines. CODESYS task configuration supports cyclic, freewheeling and event-driven execution with priorities and watchdogs. The vocabulary changes, but scheduling is the core concept.

Yael: Explain a periodic task.

Amir: Suppose a task is configured every ten milliseconds. The scheduler releases it at that interval. The task executes its assigned programs. If it completes in two milliseconds, the remaining time is available for other tasks and system services. If it occasionally needs twelve milliseconds, it has missed its intended period. Depending on the runtime, it may overlap, delay the next release, trigger a watchdog or produce a diagnostic.

Yael: A continuous task runs whenever nothing more important is ready?

Amir: Often, yes, but implementation differs. Event tasks execute because a hardware or software event occurs: a registration mark, a communication event, a high-speed input or an internal condition. They can reduce latency, but they also make timing interactions more complex. An event storm can starve lower-priority work if the architecture is careless.

Yael: Now priorities. Engineers hear “priority one” and assume it is highest, but some platforms number the opposite way.

Amir: Exactly. Never infer priority from the number without reading the platform documentation. Conceptually, a higher-priority task may pre-empt a lower-priority task. That helps a fast motion or protection-

related control function meet its deadline. But shared data then needs discipline. If a slow task updates a structure while a fast task reads it halfway through, the fast task may see an inconsistent snapshot.

Yael: How do we avoid that?

Amir: Use platform-supported data exchange mechanisms, copy coherent structures at defined boundaries, keep ownership clear, and avoid uncontrolled writes from many tasks. Some systems offer producer-consumer data, double buffers, mutex-like mechanisms or task-synchronized I/O. The exact tool varies. The design principle is deterministic ownership and transfer.

Yael: Where does the traditional input image fit?

Amir: It gives the program a coherent set of input values for an execution window. But some systems update I/O asynchronously, some modules provide immediate instructions, and networked I/O has its own update schedule. If logic assumes every tag changed at the same instant, the assumption may be false.

Yael: So the scan is a map. The task schedule is the actual timetable.

Amir: Exactly. And the timetable should be documented as part of the functional design, not discovered during a performance problem.

15:30-19:30 | Determinism, jitter and watchdogs

Yael: You keep using the word deterministic. Does it mean every cycle takes exactly the same number of microseconds?

Amir: No. It means the timing is bounded and predictable enough for the function. A temperature loop may tolerate tens or hundreds of milliseconds. Electronic camming may require far tighter synchronization. The requirement should state a maximum response or jitter, not merely “fast.”

Yael: Let us reconstruct the reject-gate chain.

Amir: First, the sensor has a response time. Then the input module samples or filters the signal. A remote I/O network transports it. The relevant task waits until its next release, executes logic, and updates an output. The output module switches. The solenoid and pneumatic cylinder move. The worst-case response is the sum of worst-case contributions plus uncertainty. Average cycle time hides the late cases that cause escapes.

Yael: And jitter is variation around the expected timing.

Amir: Yes. Task start jitter, execution-time variation, bus jitter and actuator variation. Logging minimum, average and maximum task time is more useful than a single snapshot. CPU utilization also needs margin. A system at forty percent average load can still miss deadlines if one burst consumes the high-priority task window.

Yael: What does a watchdog do?

Amir: It monitors whether a task or controller completes within a configured time or expected pattern. If the limit is exceeded, the runtime can raise a fault, stop the task, halt the application or drive outputs to configured states. CODESYS documentation, for example, describes task watchdog timing and sensitivity. The correct setting must be above legitimate worst-case execution but below the time at which loss of control becomes unacceptable.

Yael: Too tight and commissioning trips constantly. Too loose and the watchdog only reports the failure after the process is already unsafe.

Amir: Exactly. And a watchdog is not a substitute for an independent safety function. It protects execution integrity. The response to a program fault still has to be designed: which outputs de-energize, which hold, which equipment needs a controlled stop, and what independent layers remain available?

Yael: This is where software timing meets mechanical stored energy.

Amir: Pressure, inertia, gravity, heat and chemistry do not pause because the CPU faulted.

19:30-24:00 | Local I/O, remote I/O and signal paths

Yael: Now move from scheduling back to the physical signal. Why choose remote I/O instead of running every cable to the main cabinet?

Amir: Remote I/O can reduce cable length, simplify modular construction and place diagnostics near the equipment. It can also create a clean machine-module boundary. But it adds network infrastructure, power distribution and a communication failure mode. Selection is not “remote is modern.” It is a trade between wiring, maintainability, environment, latency, topology and availability.

Yael: What does the controller know when a remote rack disappears?

Amir: That depends on the platform and configuration. Inputs may be marked bad, held at last value or forced to a substitute. Outputs may hold, go to zero or apply configured fallback values. The program must use quality and connection status, not treat a stale numeric value as trustworthy. A flow value of forty can mean “forty litres per minute measured now” or “the last valid value before communication failed.” Those are different process states.

Yael: Digital I/O seems simpler.

Amir: Simpler, not simple. Input type, threshold, sink or source convention, filtering, isolation, short-circuit detection and test pulses matter. An output may be relay, transistor or triac. Leakage current can keep a sensitive load partially energized. A module can report a commanded output without proving field current or device motion. For critical actions, use independent feedback appropriate to the failure you need to detect.

Yael: Analog modules add conversion time and accuracy.

Amir: And channel configuration, wire-break detection, common-mode limits, isolation, resolution, filtering and update rate. A sixteen-bit number does not guarantee sixteen-bit system accuracy. Sensor uncertainty, wiring, reference stability, module error and scaling all contribute. We will make that the focus of Episode 3.

Yael: What about timestamps?

Amir: For sequence-of-events analysis, the time should be captured as close to the event as necessary. If the HMI timestamps an alarm after polling the PLC, two events may appear in the wrong order. Some input modules capture timestamps at the edge; some controllers synchronize clocks across a network. Again, architecture follows the diagnostic requirement.

Yael: The I/O list therefore needs more than tag name and range.

Amir: It should include electrical type, update requirement, fail behaviour, diagnostic coverage, isolation, environment, power source, cable and grounding assumptions, and how the software uses quality status.

24:00-28:00 | Memory, retention and restart behaviour

Yael: Let us simulate a power loss during the filling cycle. What should the controller remember?

Amir: That question must be answered variable by variable. Retaining everything sounds safe but can preserve an invalid state. Retaining nothing can lose production accounting, calibration or the position of material in the machine. Data should be classified by purpose and restart scenario.

Yael: Give me categories.

Amir: Configuration and calibration values usually need controlled non-volatile storage. Recipe parameters may be stored locally or restored from a recipe system. Production counters may need persistence with a defined write strategy. Temporary calculations should normally reinitialize. Sequence state is the dangerous category: after power returns, should the machine resume step seventeen, return to a safe recovery state, or require the operator to inspect and clear material?

Yael: For our filling machine, resuming automatically could double-fill a container or move the index table while someone is clearing it.

Amir: Exactly. The controller may retain encoder position, but the mechanics may have moved during loss of power. A servo absolute encoder, a homing sensor and a retained software value are three different sources of truth. The restart procedure needs validation of physical state before commands resume.

Yael: What is the difference between warm and cold restart?

Amir: Terminology varies by vendor, but generally a warm restart preserves more runtime state while a cold restart reinitializes more of the application. Never rely on the generic words alone. Test the actual platform through power loss, CPU stop-run, firmware update, project download and battery or storage failure. Document which data persists in each case.

Yael: Non-volatile memory also has write limits.

Amir: Potentially. Platforms manage this differently, but continuously writing every scan to flash is a bad assumption. Use supported retention mechanisms, buffering and transactional patterns. For important records, consider whether the controller is the authoritative database at all. A PLC is excellent at control; it is not automatically the best long-term production database.

Yael: The restart specification should be part of FAT.

Amir: Yes: interrupt power at defined stages, restore it, and verify outputs, retained values, alarms, audit logs and operator instructions. Restart behaviour is a designed operating mode, not an accident between Run and Stop.

28:00-32:00 | IEC 61131-3 and the engineering environment

Yael: Now we reach the code. What does IEC 61131-3 standardize?

Amir: It standardizes syntax and semantics for a suite of programmable-controller languages and configuration concepts. The fourth edition was published in 2025. Its official IEC overview lists Structured Text, Ladder Diagram and Function Block Diagram, with Sequential Function Chart elements used to structure programs and function blocks. Legacy projects and tools may still contain Instruction List, so engineers must distinguish current standard scope from installed-base reality.

Yael: And Program Organization Units?

Amir: Programs, functions and function blocks are key structuring units. A function normally returns a result without persistent internal instance state. A function block has instance data and is well suited to a motor, valve, PID loop or reusable equipment module. Programs organize application execution. Vendors add libraries, classes, methods, namespaces and other extensions depending on edition and platform.

Yael: Does IEC 61131-3 mean I can move a project from any vendor to another?

Amir: No. It makes concepts and languages more recognizable, but hardware configuration, libraries, data types, task models, motion blocks, safety functions, diagnostics and extensions differ. IEC 61131-10 defines an XML exchange format for projects, and PLCopen works on conformance and reusable specifications, but practical portability still requires engineering and testing.

Yael: The engineering environment also performs online monitoring and change.

Amir: Yes, and that is operationally powerful. Engineers can watch variables, force signals, modify code and download changes. Every one of those capabilities needs authorization, change control and recovery. An online change may preserve process state, but it can also create a configuration that has not followed the same FAT path as the released baseline.

Yael: So version control cannot be a zip file named final-final-two.

Amir: Correct. Store source, libraries, hardware configuration, HMI dependencies, firmware compatibility, safety signatures where applicable, and a deployable backup. Record who changed what, why, how it was

reviewed and what test evidence exists. PLCopen software construction guidelines are useful because industrial control software has the same maintainability problem as other software, with additional physical consequences.

Yael: Detailed language selection comes later?

Amir: Yes. Today the point is that the language executes inside a scheduled runtime with hardware and lifecycle constraints. Elegant Structured Text in the wrong task is still a poor control design.

32:00-36:00 | Selecting PLC, PAC or IPC

Yael: Let us select a controller for our servo-indexed filling and circulation machine. It has thirty servo axes, machine vision, eighty analogue points, recipe handling, a local HMI and plant data integration. PLC, PAC or IPC?

Amir: I would refuse to answer until we decompose the workload. How many axes need coordinated motion? What cycle time and synchronization accuracy? Does vision only return pass-fail or execute heavy image processing? What are the analogue loop periods? What availability is required? How long will the machine be supported? Who maintains it at two in the morning?

Yael: Suppose the vision workload is computationally heavy and the motion is tightly synchronized.

Amir: A PC-based control platform may be attractive because it can allocate real-time cores to control and other cores to vision or analytics. Beckhoff, for example, describes TwinCAT as a PC-based system in which real-time control runs separately from the host operating system and can use multiple cores. But that is one implementation, not a universal guarantee. We would still verify worst-case timing, storage failure recovery, patch strategy and separation between workloads.

Yael: A high-end modular PLC or PAC could also coordinate motion and process control.

Amir: Yes. It may give a more familiar maintenance model, integrated industrial I/O and long hardware lifecycle. If the machine builder has proven libraries and service capability on that platform, the total risk may be lower even if the raw processor is less open. Architecture is not a benchmark contest; it is a lifecycle decision.

Yael: Could we split it: motion controller or IPC for fast domains, PLC for process and utility control?

Amir: Certainly, but every split creates interfaces, synchronization and ownership questions. Which controller owns the machine state? What happens if one restarts? How are recipes coordinated? Can a network failure create conflicting commands? A distributed architecture is valuable when it creates fault containment or modularity, not when it merely spreads code across boxes.

Yael: And the PAC term?

Amir: Use it only after translating it into requirements: multiple deterministic tasks, large structured data, integrated Ethernet, motion, process libraries, redundancy, safety options, open protocols or whatever the project actually needs. Procurement should compare verified functions, not adjectives.

36:00-39:30 | Redundancy and high availability

Yael: Management asks for a redundant PLC because downtime is expensive. What should the engineer ask next?

Amir: Redundant what? CPU, power supply, network, remote-I/O adapter, server, engineering station, field sensor, actuator or utility? Two controllers do not remove a single power feed, one network switch or one control valve.

Yael: How does a hot-standby pair work conceptually?

Amir: One controller is primary and executes the application. The standby receives synchronized application state and monitors system health. If a qualifying failure occurs, control transfers. Schneider’s M580

documentation, for example, describes identically configured controllers, continuous health monitoring and state updates from primary to standby. But transfer time, synchronized data, supported I/O topology and behaviour during mismatch are platform-specific.

Yael: Is the transfer always bumpless?

Amir: Never assume that word without defining the process variable and acceptance limit. A CPU may switch within one scan while a communication connection takes longer to recover. An analogue output might hold, step or be recalculated by a controller with slightly different internal state. A motion system may need controlled resynchronization rather than transparent transfer.

Yael: What failure tests belong in FAT or SAT?

Amir: Loss of primary CPU, standby CPU, one power supply, one network path, one remote rack, clock synchronization, engineering connection and selected communication modules. Also configuration mismatch, failed switchover, switchback and maintenance replacement. Record process impact, alarm sequence and recovery action. High availability is proven through failure testing, not inferred from a redundancy diagram.

Yael: And common-cause failure?

Amir: Two identical controllers can share the same software defect, environmental exposure, cybersecurity credential or incorrect configuration. Redundancy improves availability against selected failures; it does not make the design invulnerable.

39:30-42:00 | Safety and cybersecurity boundaries

Yael: Can the standard PLC stop the machine safely when anything goes wrong?

Amir: It can perform normal interlocks and controlled stops, but a safety function requires a risk-based architecture and lifecycle. IEC 61131-6 defines requirements for an FS-PLC intended as the logic subsystem of an electrical, electronic or programmable-electronic safety-related system. That does not make every program in a safety PLC automatically safe. The sensors, logic, final elements, diagnostics, independence, validation and proof testing all matter.

Yael: Could standard and safety logic share one physical platform?

Amir: Some certified platforms support that with separation mechanisms, but the safety application remains governed by the relevant machine or process safety standard and vendor safety manual. Do not use convenience as evidence of independence.

Yael: Cybersecurity is another layer around the controller.

Amir: And partly inside it. NIST SP 800-82 Rev. 3 emphasizes securing operational technology while respecting performance, reliability and safety. ISA/IEC 62443 divides responsibilities among asset owners, integrators, service providers and product suppliers. For a controller, practical controls include asset inventory, network segmentation, role-based accounts, removal of default credentials, secure remote access, controlled firmware, backups, logging and disabling unused services where supported.

Yael: What about programming ports and online changes?

Amir: Treat them as powerful maintenance interfaces. Authenticate users, limit network paths, use approved engineering stations, preserve an offline recovery copy and monitor changes. A controller that can command motors and valves is not merely a data endpoint. Compromise can become torque, pressure, heat or chemical release.

Yael: But we should not patch blindly either.

Amir: Correct. OT security requires tested change windows and recovery plans. “Never update” accumulates risk; “update immediately without validation” can also create risk. The lifecycle must balance vulnerability, availability and verified operation.

42:00-44:00 | Worked task map and closing checklist

Yael: Let us finish with an illustrative task map for the filling machine.

Amir: One possible architecture could have a one-millisecond motion task synchronized to the drive network, a ten-millisecond machine-sequence task, a fifty-millisecond process-control task for flow and temperature, a one-hundred-millisecond diagnostic task, and slower HMI or data services. Those numbers are examples only. The real values come from motion physics, sensor and actuator dynamics, network schedules and measured execution time.

Yael: Data ownership would be explicit. Motion owns axis state. The sequence owns machine mode. Process control owns loop outputs. HMI requests are validated before they change commands.

Amir: Exactly. Each task has a period, priority, worst-case execution target, watchdog response, input source, output ownership and test method. Restart behaviour is defined. Remote I/O loss has a configured fallback. Critical physical feedback is independent from the command bit.

Yael: Give us the seven questions to ask before selecting a controller.

Amir: One: what physical functions and worst-case response times must be achieved? Two: what I/O types, update rates, diagnostics and environmental ratings are required? Three: what task model, motion, process and communication capacity is needed with margin? Four: what data must survive each restart condition? Five: what availability architecture and failure tests are required? Six: what safety and cybersecurity responsibilities belong inside or outside the controller? Seven: who will maintain the system, with what tools, backups, skills and vendor support over its lifetime?

Yael: That checklist changes the question from “Which PLC is fastest?” to “Which architecture can prove the required behaviour?”

Amir: Exactly. In Episode 3 we move one step outward to sensors: how pressure, flow, temperature, position and presence become trustworthy data, and how wiring, sampling, calibration and failure modes can mislead even perfect code.

Yael: Until then, remember that the controller does not see the machine. It sees signals.

Amir: And the quality, timing and meaning of those signals determine the quality of control. Thanks for listening to Control Edge.

Yael: See you in Episode 3.

7. Producer and host notes

- Keep a technically calm pace. Pause after the end-to-end response-time explanation and after the restart-state example.
- Do not turn the vendor examples into a ranking. The point is that different platforms implement common concepts differently.
- When saying “scan cycle,” immediately qualify that modern controllers may use multiple scheduled and event-driven tasks.
- Use “real time” to mean bounded and sufficiently predictable timing for the application, not merely high processor speed.
- Read the illustrative task periods as examples only; they are not universal values.
- For IEC 61131-3:2025, refer to the official edition and avoid claiming full project portability across engineering environments.
- Keep safety PLC discussion at architecture level. Detailed safety lifecycle and Performance Level/SIL calculation belong to later episodes.

8. Engineering controller-selection checklist

1. What physical functions, maximum response times and deadlines must be met?
2. What I/O types, update times, diagnostics, isolation and environmental conditions are required?
3. What task model, priorities, watchdog response and CPU margin are required?
4. Which data survives power loss, Stop-Run, warm restart and cold restart?
5. What is the availability architecture and which single points of failure remain?
6. Which functions belong to basic control, safety and independent protection layers?
7. How are users, remote access, firmware, backups, version control and change management controlled?
8. What is the maintenance, spares, licensing, vendor-support and knowledge-transfer strategy for ten to twenty years?

9. Glossary

Term	Meaning in this episode
Backplane	The electrical and communication structure linking a controller, power supply and modules in a rack or base.
Cycle time	Elapsed time for one execution of a task or controller cycle; it is not automatically the total physical response time.
Determinism	Timing behaviour that is bounded and predictable enough for the control function.
Event task	Logic executed because a defined event occurred, subject to the controller scheduling model.
FS-PLC	Functional-safety programmable logic controller intended for use as the logic subsystem of a safety-related system.
Industrial PC	Ruggedized computer platform used in industrial environments, sometimes running a real-time control runtime.
Jitter	Variation in the start time or duration of a periodic task or communication update.
PAC	Programmable Automation Controller; a widely used vendor and industry term, not a single strict universal architecture class.
POU	Program Organization Unit in IEC 61131-3, such as a program, function or function block.

Term	Meaning in this episode
Retentive data	Data intentionally preserved across a defined restart or power-loss condition.
Scan cycle	A simplified model in which inputs are read, logic executes and outputs update; actual controllers may schedule several tasks and I/O mechanisms.
Soft PLC	PLC runtime implemented in software on a general-purpose or industrial computing platform.
Watchdog	A timing monitor that detects when a task or controller fails to complete within configured limits.

10. Standards and source map

How to use this list
Standards are revised and adopted differently by country, sector and contract. Obtain the project-applicable edition and check amendments, national adoption and vendor instructions.

1. IEC 61131-1:2003, Programmable controllers - Part 1: General information - Definitions and principal functional characteristics of PLC systems. Official source

2. IEC 61131-2:2017, Industrial-process measurement and control - Programmable controllers - Part 2 - Functional and EMC requirements and verification tests for industrial control equipment including PLC and PAC. Official source

3. IEC 61131-3:2025, Programmable controllers - Part 3: Programming languages - Fourth edition of the programming-language standard; published 22 May 2025. Official source

4. IEC 61131-6:2012, Programmable controllers - Part 6: Functional safety - Requirements for an FS-PLC used as the logic subsystem of a safety-related system. Official source

5. IEC 61010-2-201:2024, Particular requirements for control equipment - Safety requirements for electrical control equipment used with IEC 61010-1. Official source

6. IEC 61131-10:2019, PLCopen XML exchange format - XML-based exchange format for IEC 61131-3 projects; not a guarantee of effortless portability. Official source

7. PLCopen Software Construction Guidelines - Coding rules, reusable libraries, SFC, object-oriented guidance and software quality metrics for industrial control. Official source

8. CODESYS Development System - Task Configuration - Official description of cyclic, freewheeling and event tasks, priorities and watchdogs. Official source

9. Rockwell Automation Studio 5000 - Tasks, Programs and Routines - Official execution-organization concepts for Logix controllers. Official source

10. Beckhoff TwinCAT - PC-based control - Official overview of real-time PLC, motion and other runtimes on industrial PC platforms. Official source

11. Schneider Electric Modicon M580 Hardware Reference Manual - Official controller, task and local/remote I/O architecture reference. Official source

12. Schneider Electric Modicon M580 Hot Standby System Guide - Official example of controller-state synchronization and high-availability architecture. Official source

13. OPC UA for PLCs based on IEC 61131-3 - Joint OPC Foundation and PLCopen information model for controller integration. Official source

14. NIST SP 800-82 Rev. 3, Guide to Operational Technology Security - OT security guidance addressing performance, reliability and safety requirements. Official source

15. ISA/IEC 62443 Series of Standards - Cybersecurity lifecycle and requirements for asset owners, integrators, service providers and product suppliers. Official source

11. Episode quality gate

- The listener can draw the controller data path from field signal to I/O, task execution and physical output.
- The episode separates task period, execution time, jitter, I/O update and total response time.
- PLC, PAC and IPC are compared through requirements rather than slogans.
- Retentive data and restart behaviour are presented as explicit design decisions.
- IEC 61131-3:2025 is introduced accurately without promising vendor-neutral source portability.
- Redundancy is described as a system architecture, not merely a second CPU.
- The ending creates a direct bridge to Episode 3 on sensors, signal quality and measurement failure.